ClearCut: Spoiler-Free Rewriting of Movie and TV Reviews

Code: https://github.com/tejeshwar20/clearcut

1. Abstract

Movie and TV reviews often reveal twists, deaths and endings, so a reader who only wants to know whether a film is worth watching can end up spoiled. ClearCut is a web application that takes one or more pasted English reviews and returns a spoiler-free version, together with a short verdict by aspect (overall impression, acting, story and writing, pacing, visuals and characters). Only the sentences that disclose plot are rewritten. Every other sentence is kept exactly as the reviewer wrote it, so the reviewer's opinion and voice are preserved.

The system is a rule-based NLP pipeline built from sentence splitting, curated spoiler cue patterns, a spaCy dependency parse, VADER sentiment and TF-IDF sentence selection. A final safety check re-scans the output and withholds it if any disclosure remains. It runs locally with no API key and responds in about 20 ms for a short review. On developer-written test cases it removed most spoilers and left harmless text unchanged.

2. Introduction

People read reviews to decide what to watch, and they want the reviewer's judgement of the acting, story, pacing and visuals. Many reviews also describe what happens: who dies, who the villain really is, how it ends. Once a reader has seen that, it cannot be undone. Spoiler tags exist on some sites, but they are optional, inconsistent and left to the reviewer, and many reviews have none.

The obvious fix is to delete flagged sentences or hide the whole review. That solves the spoiler problem but removes the opinion with it. A sentence such as "The ending, where the hero dies saving his village, was beautifully acted" contains both a spoiler and praise. Dropping it loses the praise, while keeping it reveals the death.

ClearCut takes a middle path. It detects the sentence that discloses plot, replaces only the revealed fact with a neutral phrase, and keeps the reviewer's reaction. Harmless sentences are not touched. The aspect verdict then summarises what the review says about each part of the film. The rest of this report states the problem, describes the approach, reports how it was tested (including where it failed), and lists the data used.

3. Problem Statement

Online reviews are the main way people decide what to watch, but many of them reveal plot twists, character deaths and endings. Spoiler tags are optional and depend on the reviewer, and many reviews have none. Deleting a whole flagged review, or a whole flagged sentence, also throws away the reviewer's opinion, which is the part the reader actually wants. Readers are left choosing between being spoiled and not reading reviews at all.

Problem: given one or more raw English reviews of a movie or TV show, automatically produce a version that

1. contains **no plot disclosures** (identity reveals, deaths, twists, endings);
2. **keeps the reviewer's opinion** and leaves harmless sentences exactly as written;
3. gives a short, readable **verdict by aspect** (overall impression, acting, story and writing, pacing, visuals, characters); and
4. **fails safe**: if a disclosure cannot be removed with confidence, the text is withheld rather than shown, because a leaked spoiler is the worst failure.

Challenges: spoilers are expressed in many ways ("Darth Vader is Luke's father", "it was all a simulation", "he doesn't make it") and usually share no keyword with the harmless text around them. Removing too much destroys the review, and removing too little leaks the plot. No large labelled spoiler dataset is available for this kind of rewriting, so the system must work without one and be measured honestly.

4. Solution Approach

The system is a pipeline in `pipeline.py` and `source_writer.py`:

1. **Preprocessing:** input validation, Unicode normalisation, and sentence splitting with NLTK Punkt.
2. **Spoiler detection:** curated cue patterns for identity reveals, deaths and fates, "the whole time" twists, dream or simulation twists, betrayal, faked events, endings and explicit "Spoiler:" markers. These are combined with a **spaCy dependency parse** (`en_core_web_sm`) to decide how much of a sentence to remove.
3. **Rewriting:** the reviewer's reaction is kept and the revealed fact is replaced with a neutral phrase. For example, "In the end the hero dies saving his village" becomes "The ending is best experienced without knowing what's coming."
4. **Sentence selection:** TF-IDF and cosine similarity choose diverse, non-redundant sentences when several reviews are combined.
5. **Final safety check:** the output is re-scanned and withheld if any disclosure remains.

Stack: Python, Flask, spaCy, NLTK, scikit-learn, VADER, Waitress, and a plain HTML/JS front end.

5. Experimental Results

Tests were run with `artifacts/measure.py`.

Measure	Result
Spoiler sentences leaked	**12 of 191**
Harmless sentences changed or blocked	**0 of 20**
Mixed reviews (harmless sentence, spoiler, harmless sentence) with surrounding harmless sentences intact	**176 of 200**
Latency, short review (median)	20 ms / 26 ms
Latency, 297-word review (median / max)	127 ms / 170 ms
All tests	removes most of the spoilers while it may still leave out on something

Cases that were not caught. A separate probe of tricky sentences (`artifacts/probe_spoilers.py`) and the 212 automated tests also pass. so unusual phrasings, sarcasm and implied spoilers that no rule covers can still slip through.

6. Data Source

- **User-supplied reviews:** the text pasted into the page (`data/sample_reviews.csv` is a sample). Reviews are processed in memory and are not stored.
- **Evaluation data:** `data/evaluation_examples.json` (26 synthetic labelled sentences) and the general cases in `artifacts/measure.py`.
- **External public datasets :**
  - Kaggle, "IMDB Spoiler Dataset" (Misra): IMDb reviews labelled as spoiler or not.
  - Hugging Face, `stanfordnlp/imdb`: 50,000 labelled movie reviews, usable as realistic input text.
- **Pretrained models:** spaCy `en_core_web_sm` and the VADER lexicon.

7. Project Files

File	Purpose
`app.py`	Flask web app: serves the page and the review-analysis API
`pipeline.py`	Core NLP: spoiler detection, rewriting, aspect sentiment, sentence selection, safety check
`source_writer.py`	Builds the reader-facing text from opinions using dependency parsing
`serve.py`	Production entry point (Waitress)
`evaluate.py`	Scores aspect sentiment on `data/evaluation_examples.json`
`templates/index.html`, `static/`	Front end: page, JavaScript, CSS, logo
`data/`	Sample reviews, evaluation examples, catalog files
`tests/`	212 automated tests
`artifacts/`	Measurement and spoiler-probe scripts (`measure.py`, `probe_spoilers.py`)
`requirements.txt`, `.env.example`	Dependencies and settings template